

Problem Set Solutions: Seminar 13

Hash Tables
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course: Algorithms and Data Structures I
Topic: universal hashing, dictionaries, LRU caches, randomized linear-time tricks
Style: Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. Hashing is the standard way to turn global search into expected constant-time local access. The same idea supports dictionaries, random sampling, string filters, geometric bucketing, prefix-sum lookup, and randomized proofs of expected time.

Contents

Problem 1. Universal hash families	2
Problem 2. Dictionary with random stored pair and linear listing	2
Problem 3. LRU cache	2
Problem 4. Equality of two arrays as sets or multisets	3
Problem 5. Intersection of two unsorted arrays	3
Problem 6. Longest subsequence of consecutive integers	3
Problem 7. Longest subarray with prescribed sum	4
Problem 8. Minimum window containing all characters of another string	4
Problem 9. Maximum number of collinear points	4
Problem 10. Avoiding expensive string comparisons in open addressing	5
Problem 11. Vertical axis of symmetry of a point set	5
Problem 12. Deletion in Bloom filters	5
Problem 13. Largest increasing arithmetic progression subset	6
Problem 14. Two randomized permutation facts	6
Problem 15. Tabulation hashing	7

Conventions. Expected-time statements assume a standard universal or sufficiently independent hash family. Multiplicities are stated explicitly when the problem is about multisets rather than sets.

Problem 1. Universal hash families

Solution.

(a) Write $x = qp + r$ with $0 \leq q, r < p$. For distinct x, y we get a nonzero vector $(q - q', r - r') \in \mathbb{Z}_p^2$. The collision condition is

$$a(q - q') + b(r - r') = 0 \pmod{p}.$$

Among the p^2 choices of (a, b) , exactly p satisfy this nontrivial linear equation. Thus the collision probability is $1/p$, so the family is universal.

(b) Choose a function uniformly from all maps $\{0, \dots, p-1\} \rightarrow \{0, \dots, p-2\}$. For two distinct inputs $x \neq y$, the values $h(x)$ and $h(y)$ are independent uniform elements of a range of size $p-1$. Hence

$$\Pr[h(x) = h(y)] = \frac{1}{p-1},$$

which is exactly the universal bound for this range size.

Problem 2. Dictionary with random stored pair and linear listing

Solution.

Algorithm. Store the pairs in a dynamic array V . Also maintain a hash table H mapping each key to its index in V . Lookup uses H . Insert appends to V and records the index in H . Delete key x by swapping its array cell with the last cell, updating the moved key's index in H , and popping the last cell. A random pair is obtained by choosing a uniform random index in V . Listing all pairs is just a scan of V .

Correctness. The invariant is that $H[x]$ is exactly the position of key x in V . Swap-delete preserves this invariant by changing only one moved index. Since every live pair appears once in V , uniform index sampling is uniform over pairs.

Running time. Expected $O(1)$ for insert, delete, lookup, size, and random pair. Listing takes $O(n)$.

Problem 3. LRU cache

Solution.

Algorithm. Maintain a hash table from key to a node of a doubly linked list. The list is ordered from most recently used at the head to least recently used at the tail. On lookup, return the value and move the node to the head. On insert/update, create or update the node and move it to the head. If the capacity is exceeded, remove the tail node and delete its key from the hash table.

Correctness. The list order is exactly the recency order. Every access makes the accessed key most recent, and no other relative order changes. Therefore the tail is precisely the least recently used key and is the correct eviction target.

Running time. Expected $O(1)$ per operation.

Problem 4. Equality of two arrays as sets or multisets

Solution.

Distinct elements. Insert all elements of the first array into a hash set. Then scan the second array and check that every element belongs to the set. Since the lengths are equal and all elements are distinct, this proves equality of sets.

With repetitions. Use a hash table of counters. Increment counters for the first array and decrement for the second. The arrays define the same multiset iff all final counters are zero.

Correctness. For distinct arrays, equality of finite sets of the same size is equivalent to mutual containment; the scan verifies containment of the second in the first. With repetitions, the counter of key x is exactly $\#_A(x) - \#_B(x)$, so all counters vanish iff all multiplicities coincide.

Running time. Expected $O(n)$ time and $O(n)$ memory.

Problem 5. Intersection of two unsorted arrays

Solution.

Algorithm. Insert the elements of the smaller array into a hash table. If set intersection is required, use a hash set and output each found key once. If multiset intersection is required, store multiplicities and decrement a counter whenever an element from the second array is output.

Correctness. A key is output exactly when it appears in both arrays. In the multiset version, the counter prevents outputting a key more times than it appears in the first array.

Running time. Expected $O(n)$ time and $O(n)$ memory.

Problem 6. Longest subsequence of consecutive integers

Solution.

Algorithm. Insert all array values into a hash set. For each value x , start a scan only if $x - 1$ is absent. Then count $x, x + 1, x + 2, \dots$ while the values exist. Keep the longest interval found.

Correctness. Every maximal block of consecutive integers has a unique first value x with $x - 1$ absent. The algorithm starts exactly once from that value and recovers the whole block. Therefore the maximum recovered block is the desired answer.

Running time. Each value is charged to the scan of the unique block containing it, so the expected total time is $O(n)$.

Problem 7. Longest subarray with prescribed sum

Solution.

Algorithm. Let $S_0 = 0$ and $S_i = a_1 + \dots + a_i$. A subarray $(j+1, \dots, i)$ has sum C iff $S_j = S_i - C$. Scan i from 0 to n and store in a hash table the earliest index where every prefix sum appeared. At position i , if $S_i - C$ was seen at index j , update the answer with length $i - j$.

Correctness. The prefix-sum identity is necessary and sufficient. For a fixed right endpoint i , the longest valid subarray uses the earliest previous occurrence of $S_i - C$, which the table stores.

Running time. Expected $O(n)$ time and $O(n)$ memory.

Problem 8. Minimum window containing all characters of another string

Solution.

Algorithm. Store required multiplicities of characters of s in a hash table. Use two pointers on t . Extend the right pointer until the current window contains all required multiplicities. Then move the left pointer while preserving validity, updating the best answer whenever the window is valid.

Correctness. For every right endpoint, the algorithm shrinks the left endpoint as far as possible while the window remains valid. Hence it considers the shortest valid window ending at that right endpoint. The minimum over all right endpoints is the global optimum.

Running time. Each pointer moves at most $|t|$ times. The expected time is $O(|s| + |t|)$.

Problem 9. Maximum number of collinear points

Solution.

Algorithm. For every base point P_i , build a hash table of directions to all other points. The direction from P_i to P_j is represented by

$$(dx/g, dy/g), \quad g = \gcd(|dx|, |dy|),$$

with a canonical sign convention. The largest bucket size plus one is the maximum number of points on a line through P_i . Take the maximum over i .

Correctness. For fixed P_i , points lie on the same line through P_i iff their normalized direction vectors are equal. Thus the largest direction class gives the largest collinear set containing P_i . Every line with at least two points is considered from each of its points, so the global maximum is found.

Running time. There are $O(n^2)$ pairs and expected $O(1)$ hash updates, so the expected time is $O(n^2)$.

Problem 10. Avoiding expensive string comparisons in open addressing

Solution.

Answer. Store a short independent fingerprint together with each long string. During probing, first compare the fingerprint. Compare the full strings only if the fingerprints match.

Correctness. If the fingerprints differ, the strings are certainly treated as different by the table, so the expensive comparison is unnecessary. If they match, the algorithm verifies equality by comparing the original strings, so correctness does not rely on the fingerprint being collision-free.

Running time. The number of full string comparisons becomes the number of true matches plus rare fingerprint collisions. With random fingerprints this is small in expectation.

Problem 11. Vertical axis of symmetry of a point set

Solution.

Algorithm. Let $x_{\min}$ and $x_{\max}$ be the minimum and maximum x -coordinates. If an axis exists, it must be

$$X = \frac{x_{\min} + x_{\max}}{2}.$$

Store all points in a hash set. For every point (x, y) check that $(x_{\min} + x_{\max} - x, y)$ is also present.

Correctness. A vertical reflection sends the leftmost point to a rightmost point, so the axis is forced. Once this axis is fixed, the set is symmetric iff every point's reflected partner is present. The algorithm checks exactly this condition.

Running time. Expected $O(n)$ time and $O(n)$ memory.

Problem 12. Deletion in Bloom filters

Solution.

Answer. One cannot delete a key by zeroing its bits because those bits may also be used by other keys. Zeroing them creates false negatives.

Use a counting Bloom filter: replace every bit by a counter. Insertion increments all counters selected by the key's hash functions. Deletion decrements them. Membership is reported if all corresponding counters are positive.

Correctness. Counters record how many inserted keys currently support each cell. Deleting one key removes only its own contribution and does not erase evidence left by other keys. Therefore no false negative is introduced by deleting an existing key.

Running time. The time per operation is proportional to the number of hash functions. Memory increases from bits to counters.

Problem 13. Largest increasing arithmetic progression subset

Solution.

Algorithm. Sort the numbers and store them in a hash set. For every pair $a_i < a_j$, let $d = a_j - a_i$. If $a_i - d$ is present, skip this pair because it is not the beginning of its progression. Otherwise greedily test

$$a_i, a_i + d, a_i + 2d, \dots$$

using the hash set and keep the longest progression.

Correctness. Every maximal increasing arithmetic progression has a unique first two elements. The predecessor test identifies exactly that first pair and skips all internal pairs. Starting from the first pair and repeatedly adding d recovers the entire maximal progression.

Running time. Sorting costs $O(n \log n)$. The progression scans charge each examined adjacent pair to one maximal progression, so the total number of hash lookups is $O(n^2)$. Expected time is $O(n^2)$ and memory is $O(n)$.

Problem 14. Two randomized permutation facts

Solution.

(a) Fix the value $p_i = x$. The first inner-loop iteration is always counted. To count the probability that the scan reaches distance s , none of the first $s - 1$ inspected positions may contain a value smaller than x . Among the remaining values, there are x values not smaller than x and $k - x$ smaller values. Summing the tail probabilities gives an expected contribution at most

$$1 + \frac{k - x}{x + 1} \leq \frac{k}{x}.$$

By linearity of expectation over $x = 1, \dots, k$,

$$\mathbb{E}[\text{cnt}] \leq \sum_{x=1}^k \frac{k}{x} = O(k \log k).$$

(b) Decompose the permutation into cycles and reverse every cycle in place. The difficulty is to start each cycle once using only $O(1)$ extra cells. Choose a random hash function h and call a vertex the leader of its cycle if $(h(i), i)$ is lexicographically minimum on that cycle. For each index i , test whether it is the leader by walking around its cycle; only leaders trigger an in-place reversal of the cycle. The expected total number of inspected edges is $O(n \log n)$ by the estimate from part (a), applied to the random order induced by the hash values on each cycle.

Running time. The inversion uses $O(1)$ auxiliary cells and expected $O(n \log n)$ time.

Problem 15. Tabulation hashing

Solution.

Answer. The family is parameterized by the random tables $T_1, \dots, T_c$, where each T_i maps one block value to an independent random q -bit string, and

$$h(x_1, \dots, x_c) = T_1(x_1) \oplus \dots \oplus T_c(x_c).$$

3-independence. Take at most three distinct keys. There is a coordinate in which not all their blocks are equal. Among at most three keys, some key has a block value that is unique in that coordinate. The table entry for this unique block participates only in that key's hash value, and it is an independent uniform q -bit string. Conditioning on all other table entries, that hash value is still uniform and independent. Repeating this elimination proves 3-independence.

Failure of 4-independence. For two coordinates choose distinct block values $a \neq a'$ and $b \neq b'$ and consider

$$(a, b), \quad (a, b'), \quad (a', b), \quad (a', b').$$

The xor of their four hash values is always zero, because every involved table entry appears exactly twice. Thus the four hashes satisfy a deterministic relation and are not independent.